

Tomcat (tcMAV)

SMART CONTRACT

Security Audit

Performed on Contracts:

TcMav.sol

MD5 Hash: 0549f2222b46161068f4bfe2c332de8e

TomcatLaunchVault.sol

MD5 Hash: 528fd62d3727aabc4baa931e8cf08b07

Github Commit Hash: 02dc14db80e93786138bcfa1aa4b57ad648d9825

Platform

ETH

hashlock.com.au

JULY 2023

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Standardised Checks	9
Intended Smart Contract Functions	11
Function List	12
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Audit Findings	15
Centralisation	19
Conclusion	20
Our Methodology	21
Disclaimers	23
About Hashlock	24
Appendix	25

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Tomcat team partnered with Hashlock to conduct a security audit of their protocol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Tomcat is a new protocol aiming to be "The Convex for the Maverick Ecosystem". The Tomcat team engaged Hashlock to ensure that the contracts deployed to date are secure.

Project Name: Tomcat

Contract Addresses:

TC_MAV_TOKEN: "0x0A1D3e7a21D4DAC9C58E80d51d9d34018b900E8B",

TOMCAT_LAUNCH_VAULT: "0x1b25729D844586DF07bBa086D95a0e9f39780405",

ZkSyncEra Addresses:

TC_MAV_TOKEN: "0x1fF97d18F824B63E0e587b523Ee1ef2C3955c9d4",

TOMCAT_LAUNCH_VAULT: "0x66A5A48102c99EC77c91805467183AF1CDC2ad5F"

Compiler Version: ^0.8.18

From Tomcat Documentation;

Tomcat adopts the proven strategy established by Convex and similar platforms, aiming to enhance returns and liquidity for stakers.

Tomcat's main features are:

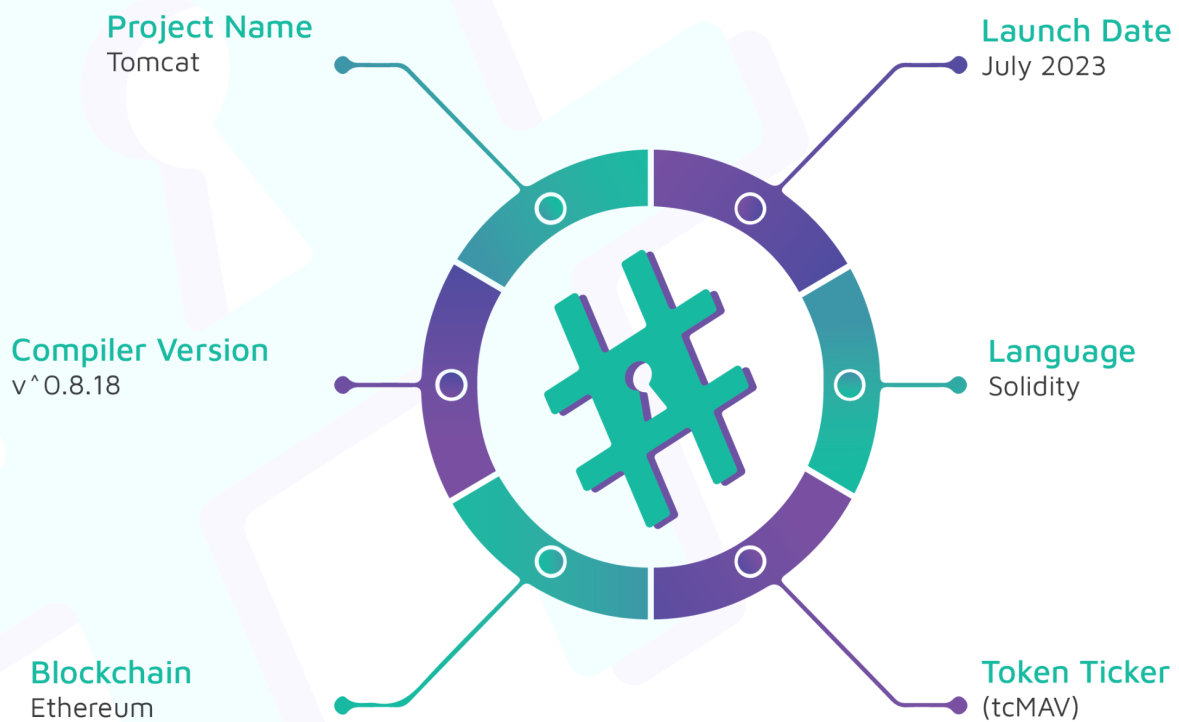
- *Maximise veMAV benefits with a liquid token*
- *Enhance rewards for Maverick LP*
- *Pass through veMAV voting via tcMAV*
- *Native Layer Zero cross-chain integration*
- *Integrated bribe market*
- *Automation options to save time & money*

Tomcat is currently live with a Launch Vault where MAV can be staked for tcMAV, giving full participation in the upcoming Governance Airdrop plus additional Tomcat rewards.

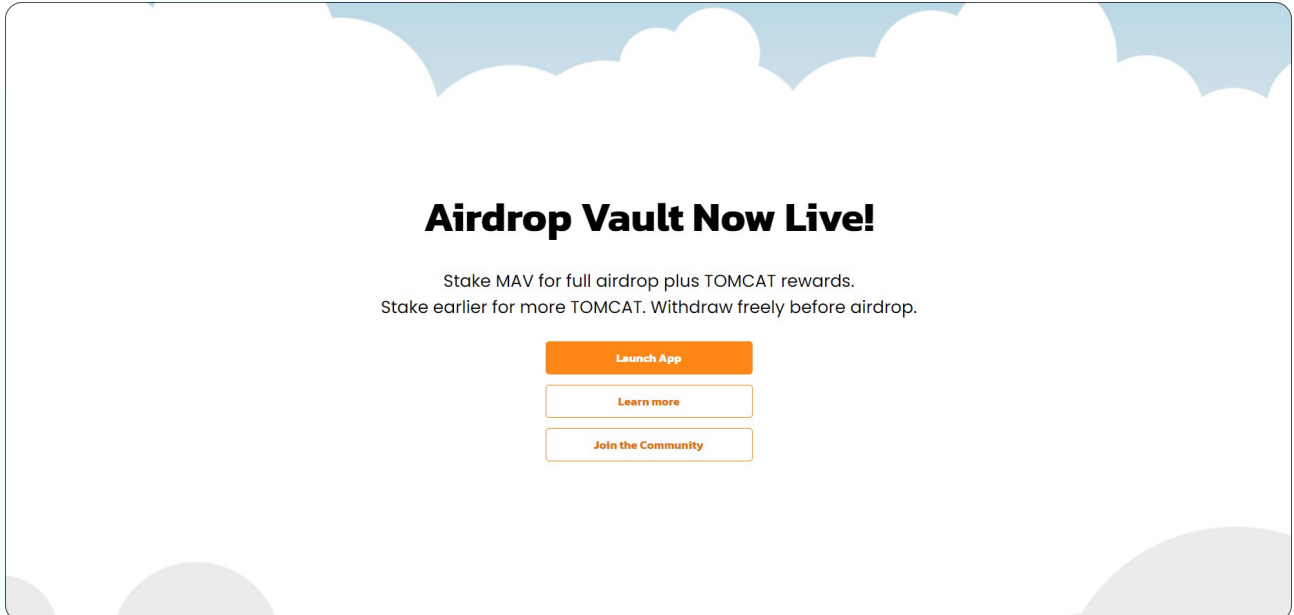
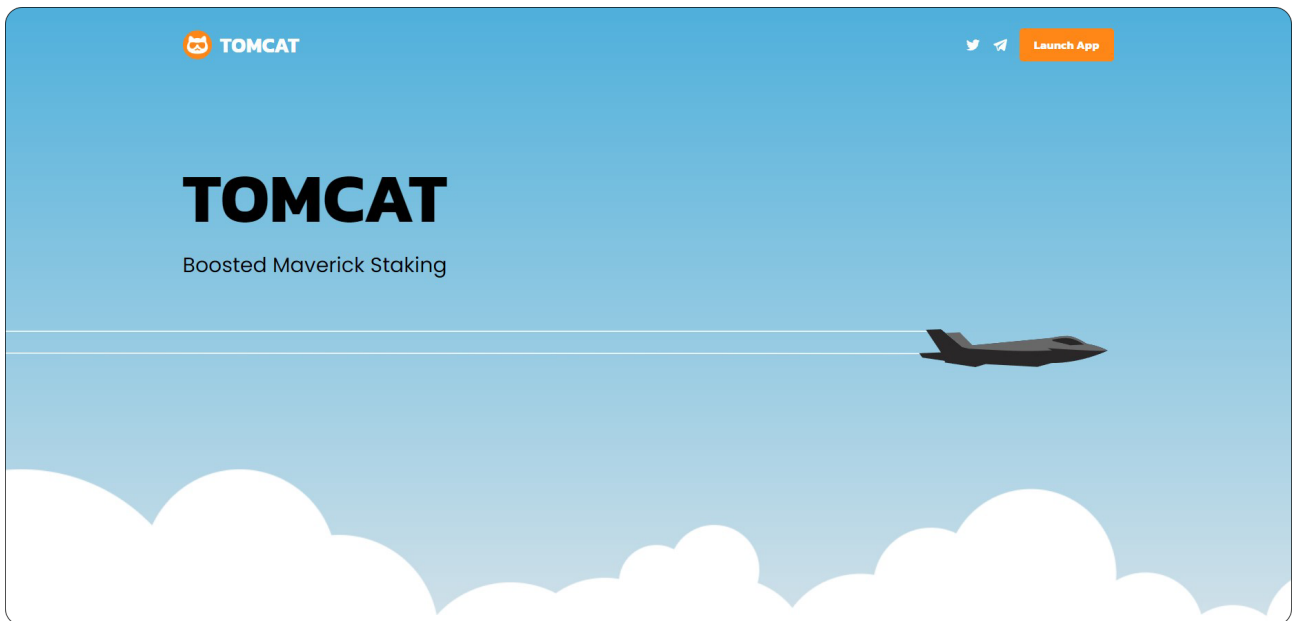
Logos:



Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Tomcat Project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	Project Review and Security Analysis Report for Tomcat Protocol Smart Contracts and other factors.
Platform	Ethereum / Solidity
Audit Date	July, 2023
Contract 1	TcMav.sol
Contract 1 MD5 Hash	0549f2222b46161068f4bfe2c332de8e
Contract 2	TomcatLaunchVault.sol
Contract 2 MD5 Hash	528fd62d3727aabc4baa931e8cf08b07
Github Commit Hash	02dc14db80e93786138bcfa1aa4b57ad648d9825

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified some low severity vulnerabilities that can improve the best practices of the protocol.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

0 High severity vulnerabilities

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Reviewed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Passed
	Function input parameters check bypass	Passed
	Correct Access control	Reviewed
	Built in emergency features	Passed
	Correct event logs	Passed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Reviewed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: PASSED



Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
File 1 TcMav.sol • Name: Tomcat tcMAV • Symbol: tcMAV <u>The owner has control over the following functions:</u> • Set whether an account can mint/burn this tcMAV token. • Creates/Destroys `amount` of tcMAV tokens and assigns them to `account`, increasing/reducing the total supply. • Current owners can transfer ownership. • Owners can renounce ownership.	Contract achieves this functionality.
File 2 TomcatLaunchVault.sol <u>The owner has control over following functions:</u> • Extend the closing time. • Set the locker contract addresses. • Current owners can transfer ownership. • Owners can renounce ownership.	Contract achieves this functionality.

Function List

TcMav.sol

	Functions	Visibility	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	onlyOwner	modifier	Passed	No Issue
3	owner	read	Passed	No Issue
4	_checkOwner	internal	Passed	No Issue
5	renounceOwnership	write	access only Owner	No Issue
6	transferOwnership	write	access only Owner	No Issue
7	_transferOwnership	internal	Passed	No Issue
8	supportsInterface	read	Passed	No Issue
9	token	read	Passed	No Issue
10	circulatingSupply	read	Passed	No Issue
11	_debitFrom	internal	Passed	No Issue
12	_creditTo	internal	Passed	No Issue
13	setMinter	external	access only Owner	No Issue
14	mint	external	Minter can mint unlimited tokens	Refer Audit Findings
15	burn	external	Minter can burn anyone's tokens	Refer Audit Findings
16	onlyMinters	modifier	Passed	No Issue

TomcatLaunchVault.sol

	Functions	Visibility	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	onlyOwner	modifier	Passed	No Issue
3	owner	read	Passed	No Issue
4	_checkOwner	internal	Passed	No Issue
5	renounceOwnership	write	access only Owner	No Issue
6	transferOwnership	write	access only Owner	No Issue
7	_transferOwnership	internal	Passed	No Issue
8	stake	external	Passed	No Issue
9	unstake	external	Passed	No Issue
10	extendClosingTimestamp	external	access only Owner	No Issue
11	setLocker	external	access only Owner	No Issue
12	lockMav	external	access only Owner	No Issue

Code Quality

This audit scope involves the solidity smart contracts of the Tomcat project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

The codebase also has a very comprehensive test coverage, which is a sign of quality code and development.

Audit Resources

We were given the Tomcat Protocol's smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

No High Severity Vulnerability was found.

Medium

[M-01] TomcatLaunchVault#setLocker - Setting the incorrect locker will irreversibly brick lockMav

The locker address can only be set once. Setting an incorrect locker address will cause the lockMav function to always revert.

Recommendation: Remove the restriction that only allows the owner to set the locker once: `if (address(locker) != address(O)) revert LockerAlreadySet();`

Status

Resolved

After communicating with the project team, it is clear that this was intentional, to give confidence that they will set the locker only once, and won't rug at the last minute before the vault lock time. The team plans to share the locker contract well before the vault lock to allow the community to review.

[M-02] TomcatLaunchVault#extendClosingTimestamp - Incorrectly extending the closingTimestamp too far will irreversibly result in missing the Maverick snapshot

extendClosingTimestamp only allows the admin to set a timestamp that's greater than the current one. If the closingTimestamp is set as after the snapshot by mistake, it cannot be reversed and the locker will not make it in the snapshot.

Recommendation: Remove this check: `if (_timestamp <= closingTimestamp) revert InvalidAmount();` (also change the function name so it accurately reflects its functionality)

Status

Resolved

Similarly to M-01, after communicating with the project team, this was also intentional along the same lines. They wish to give confidence to users that they can't bring the close time forward without them having prior notice. So they can only push the time out.

They plan to mitigate this by loading up the transaction in gnosis safe unapproved, and sharing this beforehand as our intention + when they plan on approving to give the community a chance to 'code review' the changes.

This follows best practices as exemplified by protocols such as Uniswap and MAV itself, by encoding that certain changes can only occur after a specific time.

Low / Informational

[L-01] Minter can mint and burn unlimited tokens in any address: TcMav.sol

Users with a minter role can mint and burn unlimited tokens.

Resolution: We suggest adding some limit for tokens to mint. If this is a part of the plan then disregard this issue. We also suggest changing the code so that only token holders can burn their own tokens.

Status

Resolved

With the ethos of being as immutable as practical, Tomcat made the following pragmatic decisions

There will be as much tcMAV in circulation as MAV they max lock. So the team needs to mint. Currently, Tomcat needs to burn as they offer redemptions. Maverick is yet to release their implementation for how veMAV holders can vote on LP Guages - so they need flexibility in the short term. The plan is, once Maverick 'bakes' these contracts, so will Tomcat by relinquishing ownership of the tcMAV smart contract, thereby baking in logic as to which can mint/burn and how. Finally, after surveying other LSD implementations, Tomcat's is in line with industry standard (stakeDAO and LIDO pre merge)

[L-02] Not most recent version of solidity

The contracts use solidity pragma version ^0.8.18

This is a very recent version of solidity and can be used in current production.

For best practice purposes, it is always preferable to use the absolute most recent version of solidity, which is currently ^0.8.20

Status

Acknowledged

The Tomcat team references slithers recommendation of version ^0.8.18 and has a preference to not use edge releases until there is a critical bug fix to avoid the risk of edge changes introducing critical vulnerabilities.

Gas

No Gas Optimisations were found

Centralisation

The project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

The Tomcat team plans to eventually renounce ownership of the smart contracts, making them fully decentralised.

Conclusion

After Hashlocks analysis, the Tomcat project seems to have a sound and well tested code base. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.



Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

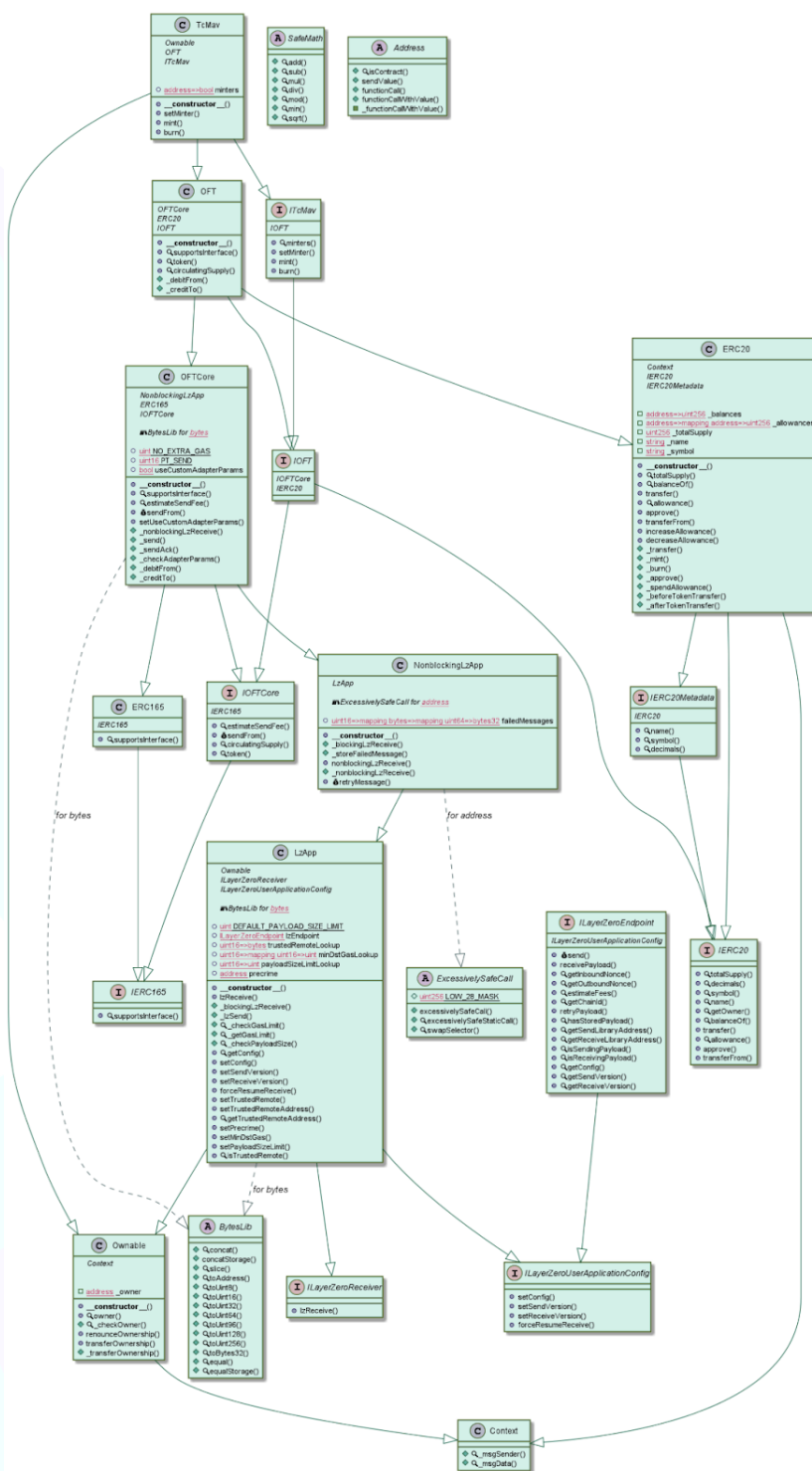
Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

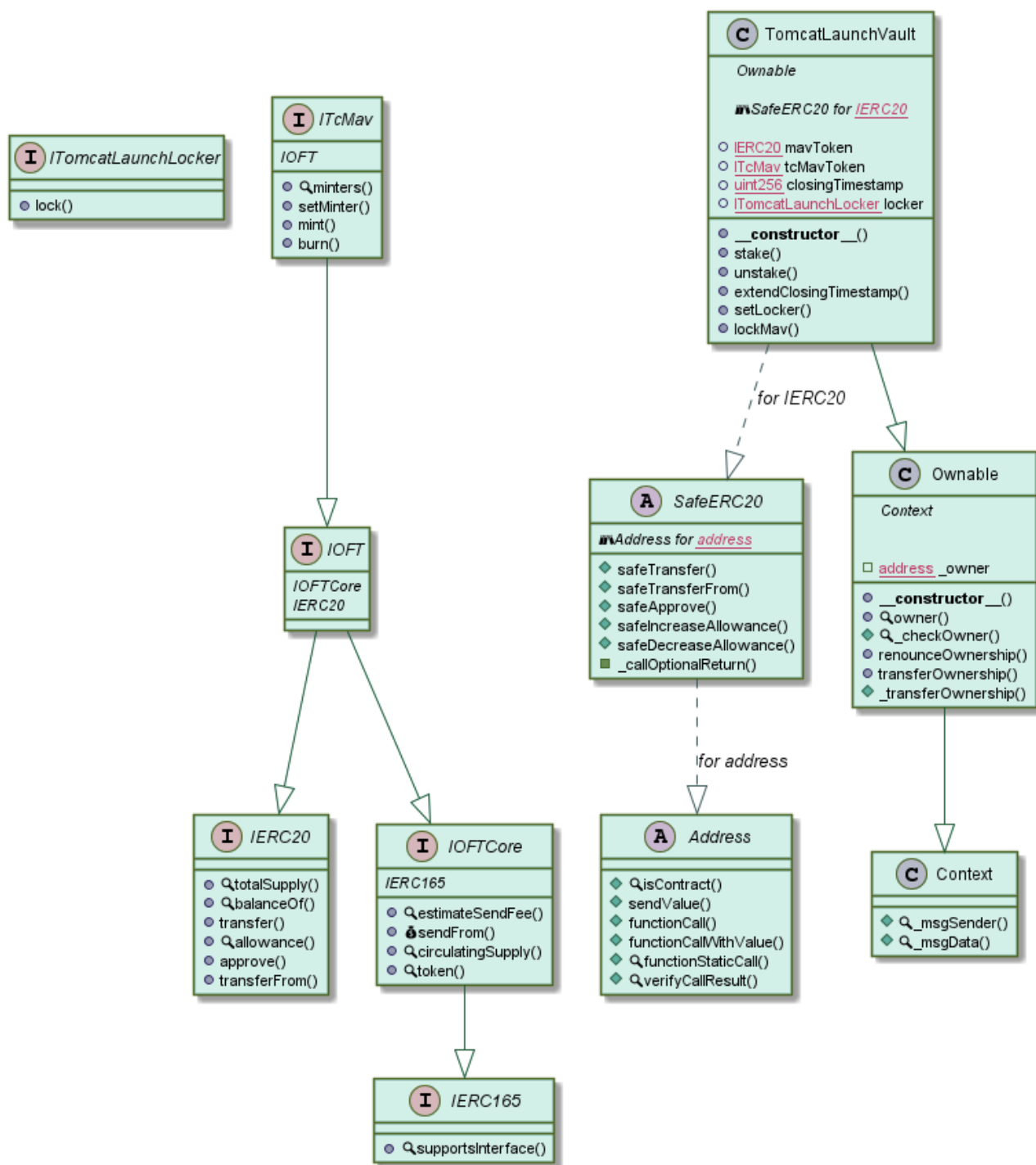
Website: hashlock.com.au

Contact: info@hashlock.com.au

TcMav Diagram



TomcatLaunchVault Diagram



Slither Results Log

Slither log >> TcMav.sol

```
OFT.constructor(string,string,address)._name (TcMav.sol#1681) shadows:
- ERC20._name (TcMav.sol#1124) (state variable)
OFT.constructor(string,string,address)._symbol (TcMav.sol#1681) shadows:
- ERC20._symbol (TcMav.sol#1125) (state variable)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#local-variable-shadowing

LzApp.setPrecrime(address)._prcrime (TcMav.sol#1525) lacks a zero-check on :
- prcrime = _prcrime (TcMav.sol#1526)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation

Variable 'BytesLib.concatStorage(bytes,bytes).sc_concatStorage_asm_0 (TcMav.sol#275)' in BytesLib.concatStorage(bytes,bytes) (TcMav.sol#219-354) potentially used before declaration: sc_concatStorage_asm_0 = keccak256(uint256,uint256)(0x0,0x20) + slen
h_concatStorage_asm_0 / 32 (TcMav.sol#323)
Variable 'BytesLib.concatStorage(bytes,bytes).submod_concatStorage_asm_0 (TcMav.sol#289)' in BytesLib.concatStorage(bytes,byte
s) (TcMav.sol#219-354) potentially used before declaration: submod_concatStorage_asm_0 = 32 - slengthmod_concatStorage_asm_0 (
TcMav.sol#332)
Variable 'BytesLib.concatStorage(bytes,bytes).submod_concatStorage_asm_0 (TcMav.sol#289)' in BytesLib.concatStorage(bytes,byte
s) (TcMav.sol#219-354) potentially used before declaration: mc_concatStorage_asm_0 = _postBytes + submod_concatStorage_asm_0 (
TcMav.sol#333)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: mc_concatStorage_asm_0 = _postBytes + submod_concatStorage_asm_0 (TcMa
v.sol#333)
Variable 'BytesLib.concatStorage(bytes,bytes).end_concatStorage_asm_0 (TcMav.sol#291)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: end_concatStorage_asm_0 = _postBytes + mlength_concatStorage_asm_0 (T
cMav.sol#334)
Variable 'BytesLib.concatStorage(bytes,bytes).mask_concatStorage_asm_0 (TcMav.sol#292)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: mask_concatStorage_asm_0 = 0x100 ** submod_concatStorage_asm_0 - 1 (
TcMav.sol#335)
Variable 'BytesLib.concatStorage(bytes,bytes).submod_concatStorage_asm_0 (TcMav.sol#289)' in BytesLib.concatStorage(bytes,byte
s) (TcMav.sol#219-354) potentially used before declaration: mask_concatStorage_asm_0 = 0x100 ** submod_concatStorage_asm_0 - 1
(TcMav.sol#335)
Variable 'BytesLib.concatStorage(bytes,bytes).mask_concatStorage_asm_0 (TcMav.sol#292)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,load(uint256)(sc_con
catStorage_asm_0) + mload(uint256)(mc_concatStorage_asm_0) & mask_concatStorage_asm_0) (TcMav.sol#337)

Variable 'BytesLib.concatStorage(bytes,bytes).sc_concatStorage_asm_0 (TcMav.sol#275)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,sload(uint256)(sc_conca
tStorage_asm_0) + mload(uint256)(mc_concatStorage_asm_0) & mask_concatStorage_asm_0) (TcMav.sol#337)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,sload(uint256)(sc_conca
tStorage_asm_0) + mload(uint256)(mc_concatStorage_asm_0) & mask_concatStorage_asm_0) (TcMav.sol#337)
Variable 'BytesLib.concatStorage(bytes,bytes).sc_concatStorage_asm_0 (TcMav.sol#275)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sc_concatStorage_asm_0 = sc_concatStorage_asm_0 + 1 (TcMav.sol#340)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: mc_concatStorage_asm_0 = mc_concatStorage_asm_0 + 0x20 (TcMav.sol#341)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: mc_concatStorage_asm_0 < end_concatStorage_asm_0 (TcMav.sol#342)
Variable 'BytesLib.concatStorage(bytes,bytes).end_concatStorage_asm_0 (TcMav.sol#291)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: mc_concatStorage_asm_0 < end_concatStorage_asm_0 (TcMav.sol#342)
Variable 'BytesLib.concatStorage(bytes,bytes).mask_concatStorage_asm_0 (TcMav.sol#292)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: mask_concatStorage_asm_0 = 0x100 ** mc_concatStorage_asm_0 - end_con
catStorage_asm_0 (TcMav.sol#349)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: mask_concatStorage_asm_0 = 0x100 ** mc_concatStorage_asm_0 - end_conca
tStorage_asm_0 (TcMav.sol#349)
Variable 'BytesLib.concatStorage(bytes,bytes).end_concatStorage_asm_0 (TcMav.sol#291)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: mask_concatStorage_asm_0 = 0x100 ** mc_concatStorage_asm_0 - end_conc
atStorage_asm_0 (TcMav.sol#349)
Variable 'BytesLib.concatStorage(bytes,bytes).mask_concatStorage_asm_0 (TcMav.sol#292)' in BytesLib.concatStorage(bytes,bytes)
(TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,mload(uint256)(mc_con
catStorage_asm_0) / mask_concatStorage_asm_0 * mask_concatStorage_asm_0) (TcMav.sol#351)
Variable 'BytesLib.concatStorage(bytes,bytes).sc_concatStorage_asm_0 (TcMav.sol#275)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,mload(uint256)(mc_conca
tStorage_asm_0) / mask_concatStorage_asm_0 * mask_concatStorage_asm_0) (TcMav.sol#351)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,mload(uint256)(mc_conca
tStorage_asm_0) / mask_concatStorage_asm_0 * mask_concatStorage_asm_0) (TcMav.sol#351)
Variable 'BytesLib.concatStorage(bytes,bytes).sc_concatStorage_asm_0 (TcMav.sol#275)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,mload(uint256)(mc_conca
tStorage_asm_0)) (TcMav.sol#346)

Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sstore(uint256,uint256)(sc_concatStorage_asm_0,mload(uint256)(mc_conca
tStorage_asm_0)) (TcMav.sol#346)
Variable 'BytesLib.concatStorage(bytes,bytes).sc_concatStorage_asm_0 (TcMav.sol#275)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: sc_concatStorage_asm_0 = sc_concatStorage_asm_0 + 1 (TcMav.sol#343)
Variable 'BytesLib.concatStorage(bytes,bytes).mc_concatStorage_asm_0 (TcMav.sol#290)' in BytesLib.concatStorage(bytes,bytes) (
TcMav.sol#219-354) potentially used before declaration: mc_concatStorage_asm_0 = mc_concatStorage_asm_0 + 0x20 (TcMav.sol#344)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#pre-declaration-usage-of-local-variables

Reentrancy in OFTCore._send(address,uint16,bytes,uint256,address,address,bytes) (TcMav.sol#1647-1656):
  External calls:
  - _lzSend(_dstChainId,lzPayload,_refundAddress,_zroPaymentAddress,_adapterParams,msg.value) (TcMav.sol#1653)
  - _lzEndpoint.send{value:_nativeFee}(_dstChainId,trustedRemote,_payload,_refundAddress,_zroPaymentAddress,_ada
pterParams) (TcMav.sol#1460)
  Event emitted after the call(s):
  - SendToChain(_dstChainId,from,toAddress,amount) (TcMav.sol#1655)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

ExcessivelySafeCall.excessivelySafeCall(address,uint256,uint16,bytes) (TcMav.sol#25-60) uses assembly
- INLINE ASM (TcMav.sol#39-58)
ExcessivelySafeCall.excessivelySafeStaticCall(address,uint256,uint16,bytes) (TcMav.sol#77-111) uses assembly
- INLINE ASM (TcMav.sol#91-109)
```

```

ExcessivelySafeCall.swapSelector(bytes4,bytes) (TcMav.sol#122-137) uses assembly
- INLINE ASM (TcMav.sol#128-136)
BytesLib.concat(bytes,bytes) (TcMav.sol#141-217) uses assembly
- INLINE ASM (TcMav.sol#151-214)
BytesLib.concatStorage(bytes,bytes) (TcMav.sol#219-354) uses assembly
- INLINE ASM (TcMav.sol#220-353)
BytesLib.slice(bytes,uint256,uint256) (TcMav.sol#356-423) uses assembly
- INLINE ASM (TcMav.sol#370-420)
BytesLib.toAddress(bytes,uint256) (TcMav.sol#425-434) uses assembly
- INLINE ASM (TcMav.sol#429-431)
BytesLib.toUint8(bytes,uint256) (TcMav.sol#436-445) uses assembly
- INLINE ASM (TcMav.sol#440-442)
BytesLib.toUint16(bytes,uint256) (TcMav.sol#447-456) uses assembly
- INLINE ASM (TcMav.sol#451-453)
BytesLib.toUint32(bytes,uint256) (TcMav.sol#458-467) uses assembly
- INLINE ASM (TcMav.sol#462-464)
BytesLib.toUint64(bytes,uint256) (TcMav.sol#469-478) uses assembly
- INLINE ASM (TcMav.sol#473-475)
BytesLib.toUint96(bytes,uint256) (TcMav.sol#480-489) uses assembly
- INLINE ASM (TcMav.sol#484-486)
BytesLib.toUint128(bytes,uint256) (TcMav.sol#491-500) uses assembly
- INLINE ASM (TcMav.sol#495-497)
BytesLib.toUint256(bytes,uint256) (TcMav.sol#502-511) uses assembly
- INLINE ASM (TcMav.sol#506-508)
BytesLib.toBytes32(bytes,uint256) (TcMav.sol#513-522) uses assembly
- INLINE ASM (TcMav.sol#517-519)
BytesLib.equal(bytes,bytes) (TcMav.sol#524-565) uses assembly
- INLINE ASM (TcMav.sol#527-562)
BytesLib.equalStorage(bytes,bytes) (TcMav.sol#567-637) uses assembly
- INLINE ASM (TcMav.sol#577-634)
Address.isContract(address) (TcMav.sol#836-843) uses assembly
- INLINE ASM (TcMav.sol#839-841)
Address._functionCallWithValue(address,bytes,uint256,string) (TcMav.sol#882-904) uses assembly
- INLINE ASM (TcMav.sol#896-899)
LzApp._getGasLimit(bytes) (TcMav.sol#1470-1475) uses assembly
- INLINE ASM (TcMav.sol#1472-1474)

```

```

LzApp._getGasLimit(bytes) (TcMav.sol#1470-1475) uses assembly
- INLINE ASM (TcMav.sol#1472-1474)
OFTCore._nonblockingLzReceive(uint16,bytes,uint64,bytes) (TcMav.sol#1634-1645) uses assembly
- INLINE ASM (TcMav.sol#1636-1638)
Reference: https://github.com/cryptic/slither/wiki/Detector-Documentation#assembly-usage

```

```

Address._functionCallWithValue(address,bytes,uint256,string) (TcMav.sol#882-904) is never used and should be removed
Address.functionCall(address,bytes) (TcMav.sol#852-854) is never used and should be removed
Address.functionCall(address,bytes,string) (TcMav.sol#856-862) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (TcMav.sol#864-870) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256,string) (TcMav.sol#872-880) is never used and should be removed
Address.isContract(address) (TcMav.sol#836-843) is never used and should be removed
Address.sendValue(address,uint256) (TcMav.sol#845-850) is never used and should be removed
BytesLib.concat(bytes,bytes) (TcMav.sol#141-217) is never used and should be removed
BytesLib.concatStorage(bytes,bytes) (TcMav.sol#219-354) is never used and should be removed
BytesLib.equal(bytes,bytes) (TcMav.sol#524-565) is never used and should be removed
BytesLib.equalStorage(bytes,bytes) (TcMav.sol#567-637) is never used and should be removed
BytesLib.toBytes32(bytes,uint256) (TcMav.sol#513-522) is never used and should be removed
BytesLib.toUint128(bytes,uint256) (TcMav.sol#491-500) is never used and should be removed
BytesLib.toUint16(bytes,uint256) (TcMav.sol#447-456) is never used and should be removed
BytesLib.toUint256(bytes,uint256) (TcMav.sol#502-511) is never used and should be removed
BytesLib.toUint32(bytes,uint256) (TcMav.sol#458-467) is never used and should be removed
BytesLib.toUint64(bytes,uint256) (TcMav.sol#469-478) is never used and should be removed
BytesLib.toUint8(bytes,uint256) (TcMav.sol#436-445) is never used and should be removed
BytesLib.toUint96(bytes,uint256) (TcMav.sol#480-489) is never used and should be removed
Context._msgData() (TcMav.sol#941-944) is never used and should be removed
ExcessivelySafeCall.excessivelySafeStaticCall(address,uint256,uint16,bytes) (TcMav.sol#77-111) is never used and should be removed
ExcessivelySafeCall.swapSelector(bytes4,bytes) (TcMav.sol#122-137) is never used and should be removed
SafeMath.add(uint256,uint256) (TcMav.sol#757-762) is never used and should be removed
SafeMath.div(uint256,uint256) (TcMav.sol#790-792) is never used and should be removed
SafeMath.div(uint256,uint256,string) (TcMav.sol#794-803) is never used and should be removed
SafeMath.min(uint256,uint256) (TcMav.sol#818-820) is never used and should be removed
SafeMath.mod(uint256,uint256) (TcMav.sol#805-807) is never used and should be removed
SafeMath.mod(uint256,uint256,string) (TcMav.sol#809-816) is never used and should be removed
SafeMath.mul(uint256,uint256) (TcMav.sol#779-788) is never used and should be removed
SafeMath.sqrt(uint256) (TcMav.sol#822-833) is never used and should be removed

```

```

SafeMath.sqrt(uint256) (TcMav.sol#822-833) is never used and should be removed
SafeMath.sub(uint256,uint256) (TcMav.sol#764-766) is never used and should be removed
SafeMath.sub(uint256,uint256,string) (TcMav.sol#768-777) is never used and should be removed
Reference: https://github.com/cryptic/slither/wiki/Detector-Documentation#dead-code

```

```

Pragma version0.8.18 (TcMav.sol#1) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6/0.8.1
6
solc-0.8.18 is not recommended for deployment
Reference: https://github.com/cryptic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

```

```

Low level call in Address.sendValue(address,uint256) (TcMav.sol#845-850):
- (success) = recipient.call{value: amount}() (TcMav.sol#848)
Low level call in Address._functionCallWithValue(address,bytes,uint256,string) (TcMav.sol#882-904):
- (success,returndata) = target.call{value: weiValue}(data) (TcMav.sol#890)
Reference: https://github.com/cryptic/slither/wiki/Detector-Documentation#low-level-calls

```

```

Parameter ExcessivelySafeCall.excessivelySafeCall(address,uint256,uint16,bytes)._target (TcMav.sol#26) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeCall(address,uint256,uint16,bytes)._gas (TcMav.sol#27) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeCall(address,uint256,uint16,bytes)._maxCopy (TcMav.sol#28) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeCall(address,uint256,uint16,bytes)._calldata (TcMav.sol#29) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeStaticCall(address,uint256,uint16,bytes)._target (TcMav.sol#78) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeStaticCall(address,uint256,uint16,bytes)._gas (TcMav.sol#79) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeStaticCall(address,uint256,uint16,bytes)._maxCopy (TcMav.sol#80) is not in mixedCase
Parameter ExcessivelySafeCall.excessivelySafeStaticCall(address,uint256,uint16,bytes)._calldata (TcMav.sol#81) is not in mixedCase

```



```

Parameter ExcessivelySafeCall.swapSelector(bytes4,bytes)._newSelector (TcMav.sol#122) is not in mixedCase
Parameter ExcessivelySafeCall.swapSelector(bytes4,bytes)._buf (TcMav.sol#122) is not in mixedCase
Parameter BytesLib.concat(bytes,bytes)._preBytes (TcMav.sol#142) is not in mixedCase
Parameter BytesLib.concat(bytes,bytes)._postBytes (TcMav.sol#143) is not in mixedCase
Parameter BytesLib.concatStorage(bytes,bytes)._preBytes (TcMav.sol#219) is not in mixedCase
Parameter BytesLib.concatStorage(bytes,bytes)._postBytes (TcMav.sol#219) is not in mixedCase
Parameter BytesLib.slice(bytes,uint256,uint256)._bytes (TcMav.sol#357) is not in mixedCase
Parameter BytesLib.slice(bytes,uint256,uint256)._start (TcMav.sol#358) is not in mixedCase
Parameter BytesLib.slice(bytes,uint256,uint256)._length (TcMav.sol#359) is not in mixedCase
Parameter BytesLib.toAddress(bytes,uint256)._bytes (TcMav.sol#425) is not in mixedCase
Parameter BytesLib.toAddress(bytes,uint256)._start (TcMav.sol#425) is not in mixedCase
Parameter BytesLib.toUint8(bytes,uint256)._bytes (TcMav.sol#436) is not in mixedCase
Parameter BytesLib.toUint8(bytes,uint256)._start (TcMav.sol#436) is not in mixedCase
Parameter BytesLib.toUint16(bytes,uint256)._bytes (TcMav.sol#447) is not in mixedCase
Parameter BytesLib.toUint16(bytes,uint256)._start (TcMav.sol#447) is not in mixedCase
Parameter BytesLib.toUint32(bytes,uint256)._bytes (TcMav.sol#458) is not in mixedCase
Parameter BytesLib.toUint32(bytes,uint256)._start (TcMav.sol#458) is not in mixedCase
Parameter BytesLib.toUint64(bytes,uint256)._bytes (TcMav.sol#469) is not in mixedCase
Parameter BytesLib.toUint64(bytes,uint256)._start (TcMav.sol#469) is not in mixedCase
Parameter BytesLib.toUint96(bytes,uint256)._bytes (TcMav.sol#480) is not in mixedCase
Parameter BytesLib.toUint96(bytes,uint256)._start (TcMav.sol#480) is not in mixedCase
Parameter BytesLib.toUint128(bytes,uint256)._bytes (TcMav.sol#491) is not in mixedCase
Parameter BytesLib.toUint128(bytes,uint256)._start (TcMav.sol#491) is not in mixedCase
Parameter BytesLib.toUint256(bytes,uint256)._bytes (TcMav.sol#502) is not in mixedCase
Parameter BytesLib.toUint256(bytes,uint256)._start (TcMav.sol#502) is not in mixedCase
Parameter BytesLib.toBytes32(bytes,uint256)._bytes (TcMav.sol#513) is not in mixedCase
Parameter BytesLib.toBytes32(bytes,uint256)._start (TcMav.sol#513) is not in mixedCase
Parameter BytesLib.equal(bytes,bytes)._preBytes (TcMav.sol#524) is not in mixedCase
Parameter BytesLib.equal(bytes,bytes)._postBytes (TcMav.sol#524) is not in mixedCase
Parameter BytesLib.equalStorage(bytes,bytes)._preBytes (TcMav.sol#568) is not in mixedCase
Parameter BytesLib.equalStorage(bytes,bytes)._postBytes (TcMav.sol#569) is not in mixedCase
Parameter LzApp.lzReceive(uint16,bytes,uint64,bytes)._srcChainId (TcMav.sol#1442) is not in mixedCase
Parameter LzApp.lzReceive(uint16,bytes,uint64,bytes)._srcAddress (TcMav.sol#1442) is not in mixedCase
Parameter LzApp.lzReceive(uint16,bytes,uint64,bytes)._nonce (TcMav.sol#1442) is not in mixedCase
Parameter LzApp.lzReceive(uint16,bytes,uint64,bytes)._payload (TcMav.sol#1442) is not in mixedCase
Parameter LzApp.getConfig(uint16,uint16,address,uint256)._version (TcMav.sol#1486) is not in mixedCase
Parameter LzApp.getConfig(uint16,uint16,address,uint256)._chainId (TcMav.sol#1486) is not in mixedCase

Parameter OFTCore.sendFrom(address,uint16,bytes,uint256,address,address,bytes)._toAddress (TcMav.sol#1625) is not in mixedCase
Parameter OFTCore.sendFrom(address,uint16,bytes,uint256,address,address,bytes)._amount (TcMav.sol#1625) is not in mixedCase
Parameter OFTCore.sendFrom(address,uint16,bytes,uint256,address,address,bytes)._refundAddress (TcMav.sol#1625) is not in mixedCase
Parameter OFTCore.sendFrom(address,uint16,bytes,uint256,address,address,bytes)._zroPaymentAddress (TcMav.sol#1625) is not in mixedCase
Parameter OFTCore.sendFrom(address,uint16,bytes,uint256,address,address,bytes)._adapterParams (TcMav.sol#1625) is not in mixedCase
Parameter OFTCore.setUseCustomAdapterParams(bool)._useCustomAdapterParams (TcMav.sol#1629) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions

Redundant expression "this (TcMav.sol#942)" inContext (TcMav.sol#936-945)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#redundant-statements

ExcessivelySafeCall.slitherConstructorConstantVariables() (TcMav.sol#6-138) uses literals with too many digits:
- LOW_28_MASK = 0x00000000ffffffffffffffffffffffffffffffffffffffffffffffff (TcMav.sol#7-8)
BytesLib.toAddress(bytes,uint256) (TcMav.sol#425-434) uses literals with too many digits:
- tempAddress = mload(uint256)( _bytes + 0x20 + _start) / 0x100000000000000000000000000000000 (TcMav.sol#430)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#too-many-digits

TcMav (TcMav.sol#1710-1756) does not implement functions:
- IERC20Metadata.decimals() (TcMav.sol#961)
- IERC20.getOwner() (TcMav.sol#915)
- IERC20Metadata.name() (TcMav.sol#951)
- IERC20Metadata.symbol() (TcMav.sol#956)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unimplemented-functions
TcMav.sol analyzed (22 contracts with 84 detectors), 181 result(s) found

```

Slither log >> TomcatLaunchVault.sol

```

Address.verifyCallResult(bool,bytes,string) (TomcatLaunchVault.sol#284-304) uses assembly
- INLINE ASM (TomcatLaunchVault.sol#296-299)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage

Address.functionCall(address,bytes) (TomcatLaunchVault.sol#168-170) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (TomcatLaunchVault.sol#197-203) is never used and should be removed
Address.functionStaticCall(address,bytes) (TomcatLaunchVault.sol#230-232) is never used and should be removed
Address.functionStaticCall(address,bytes,string) (TomcatLaunchVault.sol#240-249) is never used and should be removed
Address.sendValue(address,uint256) (TomcatLaunchVault.sol#143-148) is never used and should be removed
Context._msgData() (TomcatLaunchVault.sol#477-480) is never used and should be removed
SafeERC20.safeApprove(IERC20,address,uint256) (TomcatLaunchVault.sol#335-348) is never used and should be removed
SafeERC20.safeDecreaseAllowance(IERC20,address,uint256) (TomcatLaunchVault.sol#359-370) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code

Pragma version0.8.18 (TomcatLaunchVault.sol#1) necessitates a version too recent to be trusted. Consider deploying with 0.6.12
/0.7.6/0.8.16
solc-0.8.18 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in Address.sendValue(address,uint256) (TomcatLaunchVault.sol#143-148):
- (success) = recipient.call{value: amount}{} (TomcatLaunchVault.sol#146)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (TomcatLaunchVault.sol#211-222):
- (success, returndata) = target.call{value: value}(data) (TomcatLaunchVault.sol#220)
Low level call in Address.functionStaticCall(address,bytes,string) (TomcatLaunchVault.sol#240-249):
- (success, returndata) = target.staticcall(data) (TomcatLaunchVault.sol#247)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Parameter TomcatLaunchVault.stake(uint256)._amount (TomcatLaunchVault.sol#599) is not in mixedCase
Parameter TomcatLaunchVault.unstake(uint256)._amount (TomcatLaunchVault.sol#613) is not in mixedCase
Parameter TomcatLaunchVault.extendClosingTimestamp(uint256)._timestamp (TomcatLaunchVault.sol#627) is not in mixedCase
Parameter TomcatLaunchVault.setLocker(address)._locker (TomcatLaunchVault.sol#637) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions

Redundant expression "this (TomcatLaunchVault.sol#478)" inContext (TomcatLaunchVault.sol#472-481)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#redundant-statements
TomcatLaunchVault.sol analyzed (11 contracts with 84 detectors), 19 result(s) found

```

Solidity Static Analysis

TcMav.sol

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 1049:8:

This on local calls:

Use of "this" for local functions: Never use "this" to call functions in the same contract, it only consumes more gas than normal local calls.

[more](#)

Pos: 982:119:

Constant/View/Pure functions:

OFT.supportsInterface(bytes4) : Is constant but potentially should not be. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 1096:4:

No return:

OFTCore._creditTo(uint16,address,uint256): Defines a return type but never explicitly returns a value.

Pos: 1090:4:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 1084:12:

TomcatLaunchVault.sol**Check-effects-interaction:**

Potential violation of Checks-Effects-Interaction pattern in TomcatLaunchVault.lockMav(): Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 207:4:

Block timestamp:

Use of "block.timestamp": "block.timestamp" can be influenced by miners to a certain degree. That means that a miner can "choose" the block.timestamp, to a certain degree, to change the outcome of a transaction in the mined block.

[more](#)

Pos: 208:12:

Constant/View/Pure functions:

ITcMav.burn(address,uint256) : Potentially should be constant/view/pure but is not. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 100:4:

No return:

IOFTCore.token(): Defines a return type but never explicitly returns a value.

Pos: 54:4:

No return:

ITcMav.minters(address): Defines a return type but never explicitly returns a value.

Pos: 83:4:

Solhint Linter

TcMav.sol

TcMav.sol: 1:0: Compiler version 0.8.18 does not satisfy the ^0.5.8 semver requirement

TcMav.sol: 1:4: import of path @openzeppelin/contracts/access/Ownable.sol is not allowed. Specify names to import individually or bind all exports of the module into a name (import "path" as Name)

TcMav.sol: 1:5: global import of path @openzeppelin/contracts/token/ERC20/IERC20.sol is not allowed. Specify names to import individually or bind all exports of the module into a name (import "path" as Name)

TcMav.sol: 1:6: global import of path @openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol is not allowed. Specify names to import individually or bind all exports of the module into a name (import "path" as Name)

TcMav.sol: 1:8: global import of path @openzeppelin/contracts/utils/introspection/IERC165.sol is not allowed. Specify names to import individually or bind all exports of the module into a name (import "path" as Name)

TcMav.sol: 1:10: global import of path @openzeppelin/contracts/utils/introspection/ERC165.sol is not allowed. Specify names to import individually or bind all exports of the module into a name (import "path" as Name)

TcMav.sol: 1:11: global import of path @openzeppelin/contracts/token/ERC20/ERC20.sol is not allowed. Specify names to import individually or bind all exports of the module into a name (import "path" as Name)

TcMav.sol: 5:13: Explicitly mark visibility of state

TcMav.sol: 9:45: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:97: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:132: Provide an error message for require

TcMav.sol: 9:134: Avoid to use inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:157: Avoid to use inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:226: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 21:337: Variable "mlengthmod" is unused

TcMav.sol: 9:376: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:435: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:446: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:457: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:468: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:479: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:490: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:501: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:512: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:523: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:533: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 9:583: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 1:807: Code contains empty blocks

TcMav.sol: 5:859: Explicitly mark visibility in function (Set ignoreConstructors to true if using solidity >=0.7.0)

TcMav.sol: 9:869: Error message for require is too long

TcMav.sol: 9:879: Error message for require is too long

TcMav.sol: 9:881: Check result of "send" call

TcMav.sol: 9:893: Avoid using inline assembly. It is acceptable only in rare cases

TcMav.sol: 5:972: Explicitly mark visibility in function (Set ignoreConstructors to true if using solidity >=0.7.0)

```

TcMav.sol: 53:972: Code contains empty blocks
TcMav.sol: 9:995: Error message for require is too long
TcMav.sol: 9:1005: Error message for require is too long
TcMav.sol: 9:1006: Error message for require is too long
TcMav.sol: 5:1025: Explicitly mark visibility in function (Set ignoreConstructors to true if using
solidity >=0.7.0)
TcMav.sol: 68:1025: Code contains empty blocks
TcMav.sol: 9:1048: Avoid using inline assembly. It is acceptable only in rare cases
TcMav.sol: 13:1083: Error message for require is too long
TcMav.sol: 5:1093: mark visibility in function (Set ignoreConstructors to true if using solidity
>=0.7.0)
TcMav.sol: 125:1093: Code contains empty blocks
TcMav.sol: 5:1127: Explicitly mark visibility in function (Set ignoreConstructors to true if using
solidity >=0.7.0)

```

TomcatLaunchVault.sol

```

TomcatLaunchVault.sol: 1:0: Compiler version 0.8.18 does not satisfy the ^0.5.8 semver
requirement
TomcatLaunchVault.sol: 1:4: global import of path
@openzeppelin/contracts/access/Ownable.sol is not allowed. Specify names to import
individually or bind all exports of the module into a name (import "path" as Name)
TomcatLaunchVault.sol: 1:5: global import of path
@openzeppelin/contracts/token/ERC20/IERC20.sol is not allowed. Specify names to import
individually or bind all exports of the module into a name (import "path" as Name)
TomcatLaunchVault.sol: 1:6: global import of path
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol is not allowed. Specify names to
import individually or bind all exports of the module into a name (import "path" as Name)
TomcatLaunchVault.sol: 1:8: global import of path
@openzeppelin/contracts/utils/introspection/IERC165.sol is not allowed. Specify names to
import individually or bind all exports of the module into a name (import "path" as Name)
TomcatLaunchVault.sol: 1:10: global import of path
@openzeppelin/contracts/utils/introspection/ERC165.sol is not allowed. Specify names to
import individually or bind all exports of the module into a name (import "path" as Name)
TomcatLaunchVault.sol: 1:69: Code contains empty blocks
TomcatLaunchVault.sol: 5:140: Explicitly mark visibility in function (Set ignoreConstructors to
true if using solidity >=0.7.0)
TomcatLaunchVault.sol: 13:153: Avoid making time-based decisions in your business logic
TomcatLaunchVault.sol: 13:170: Avoid making time-based decisions in your business logic
TomcatLaunchVault.sol: 13:207: Avoid making time-based decisions in your business logic

```

Software analysis result:

The software reported many false positive results and informational issues. These issues however can be safely ignored.



#Hashlock.